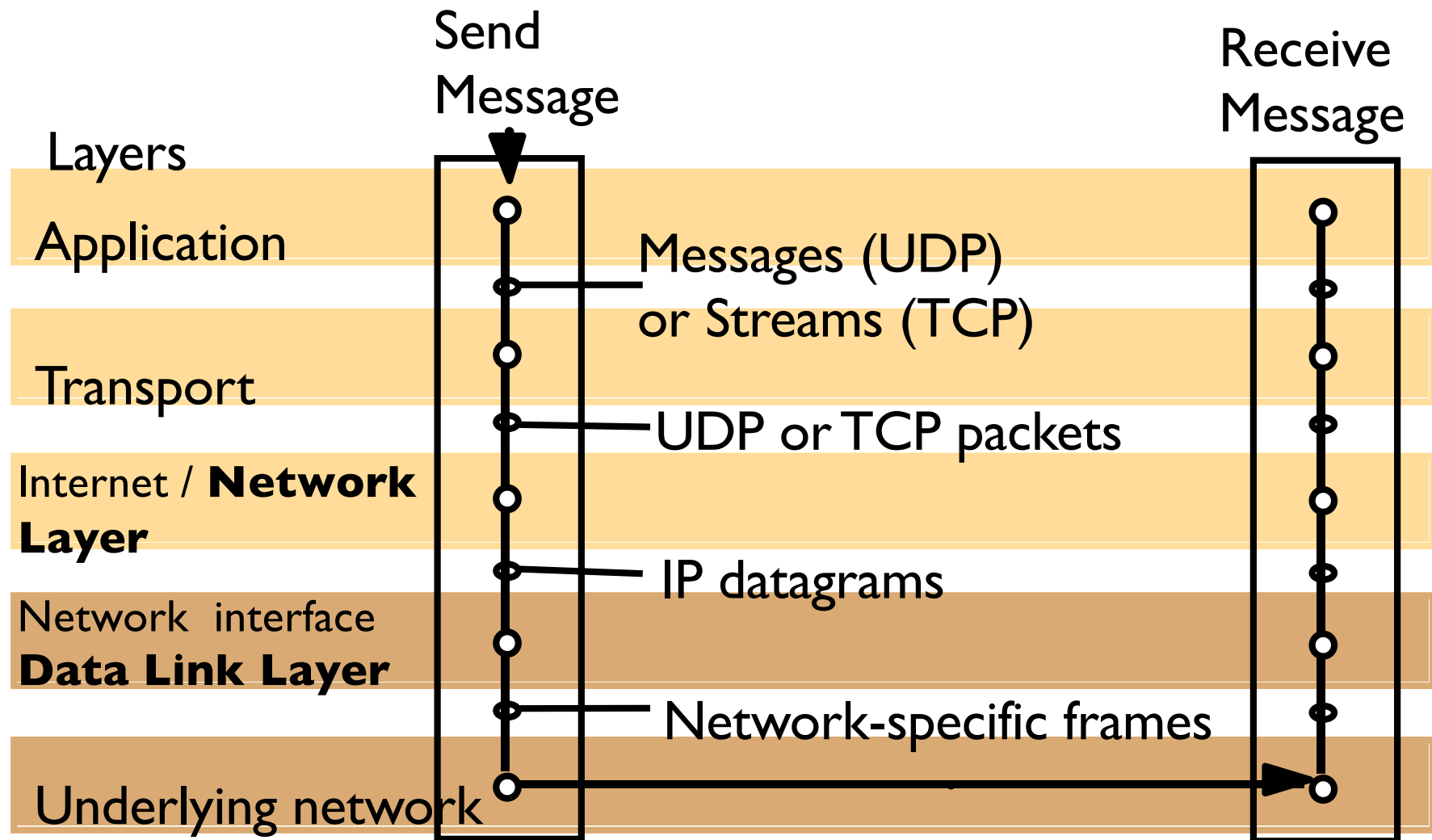

The Network Stack

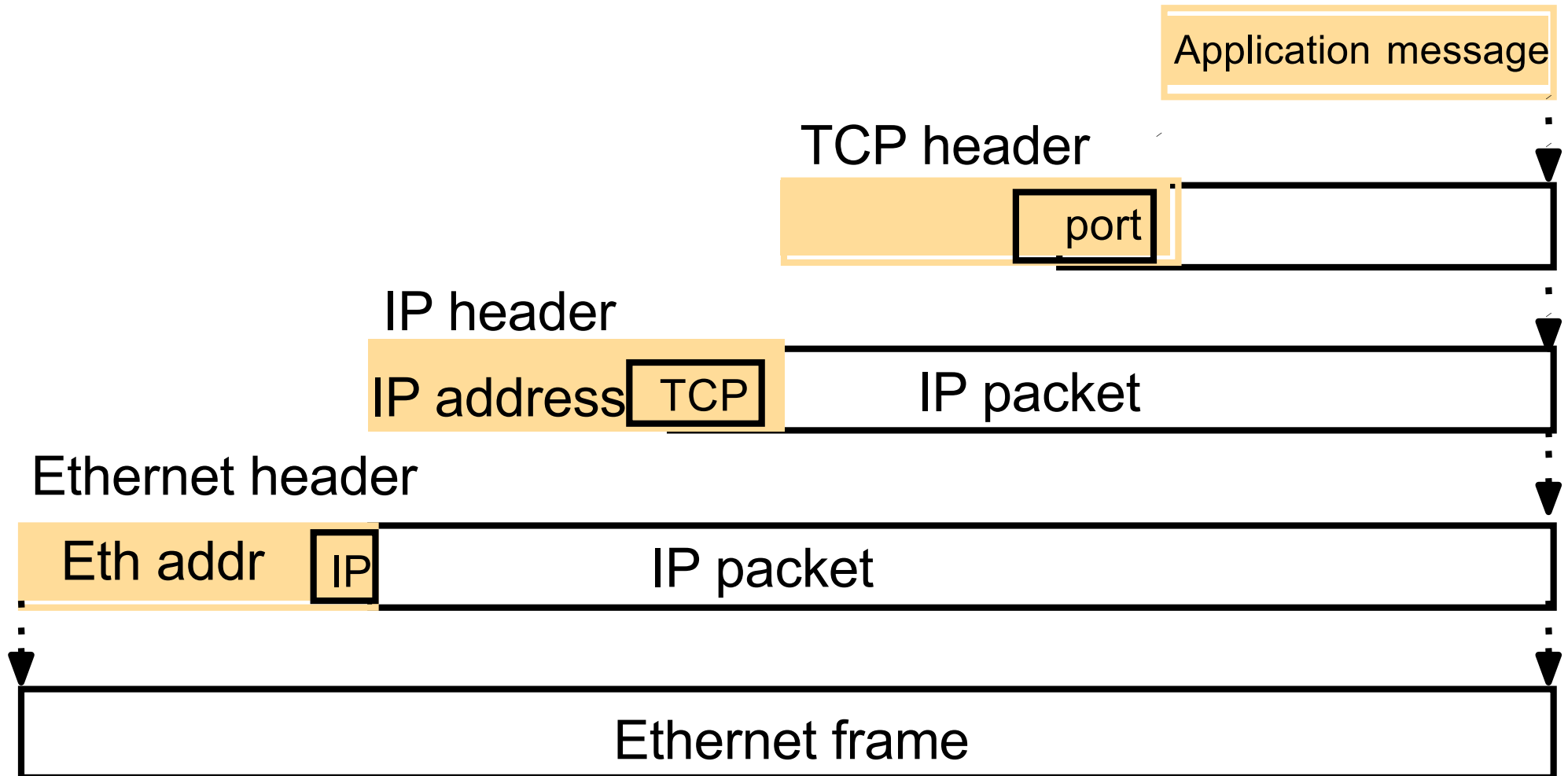
Communication Layers: Example TCP/IP



Communication Layers

- ❑ Each CL implements a “*protocol*”.
 - ☆ An algorithm (program, software component) at sender/receiver that performs the steps necessary to provide a certain functionality
 - ☆ The protocol performs certain steps
 - Upon receiving a request from the upper layer
 - Upon receiving a message from the underlying layer / the network hardware

Message Format



Data Link layer: Ethernet

- ❑ Task: Send a message from node A to node B in the same subnet
- ❑ Network addresses (physical addresses used by Ethernet – also referred as MAC addresses)
 - ☆ Upper layer must provide the correct MAC addresses
- ❑ Ethernet frame header: sender/destination (network addresses), ...
- ❑ Old: Subnet is a “Ethernet bus”:
 - ☆ many nodes connected to the same cable
 - ☆ Communication from node A to node B
 - broadcast frame on medium
 - controller hardware at each node listens on medium
 - pick up frame that are addressed to them and gives them to upper layer
 - ☆ Collision within subnet possible
 - resubmit after waiting certain time
 - How do you know that there is a collision?
 - How do you avoid repeated collisions?

Data Link layer: Ethernet

- ❑ Current reality: Ethernet “star scheme”. All nodes are connected to a switch – each has its own cable

- ☆ Communication from A to B

- A sends message to switch
 - Switch forwards to B
 - No collisions

- ☆ Routing table at switch learned on-the-fly:

- Upon first message received from a node: keep track of sender network address
 - If receiver address not known broadcast to all; actual receiver will confirm

- ❑ Wireless Ethernet: Only one medium

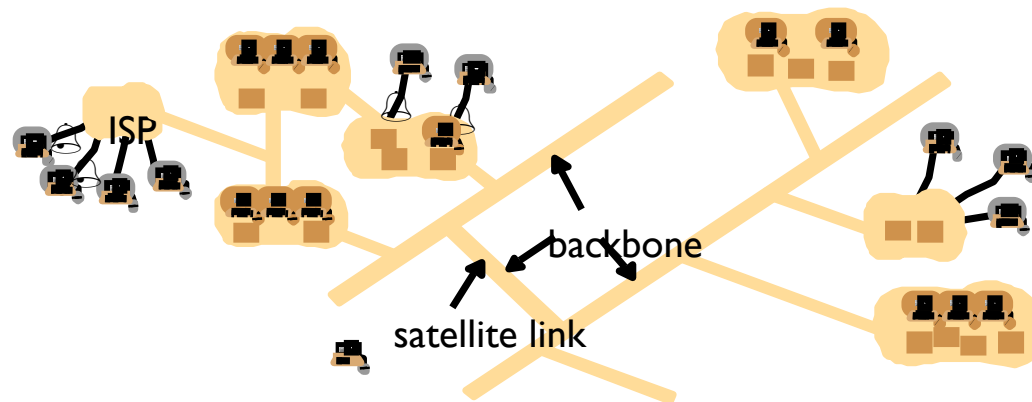
- ☆ Broadcast unavoidable

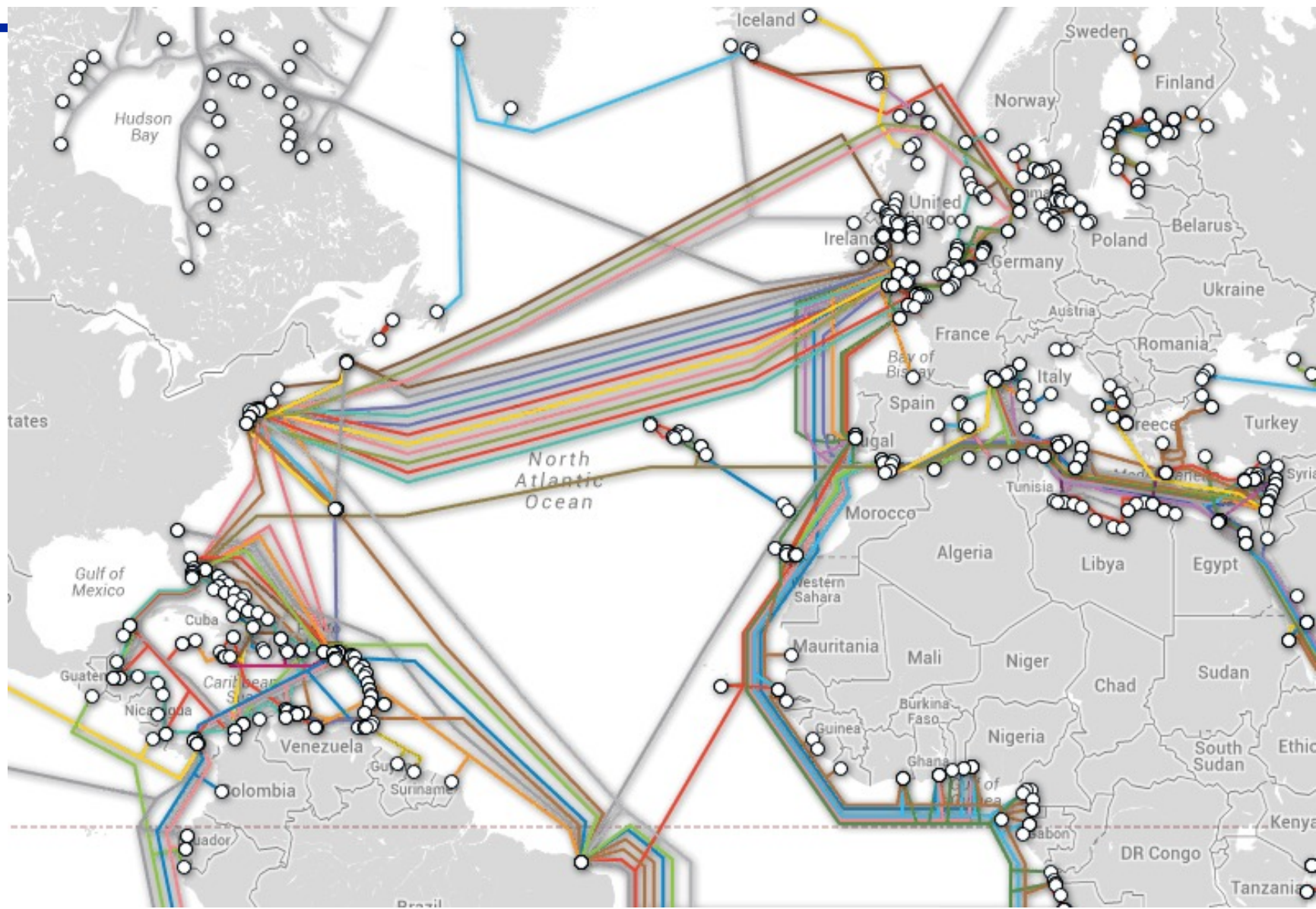
- ☆ Collisions difficult to detect

- ☆ Broadcast reservation scheme

Network Layer: IP protocol

- ❑ host-to-host transmission
- ❑ uses IP address
- ❑ unreliable, best-effort
 - ☆ Packets can get lost somewhere
- ❑ Tasks:
 - ☆ transform IP address to network address of underlying network protocol
 - ☆ ROUTING:
 - ☆ special routers in a LAN are connected to neighboring LANs or to backbone link





<https://www.vox.com/a/internet-maps>

Addresses

□ IP Addresses: 32 bits

★ Network part: identifies subnet

★ Host part: identifies host on subnet

★ Hierarchical assignment

- Organization is given a subnet identifier
- Organization assigns host identifiers within subnet

Addresses

- ❑ Routers/Bridges connect different subnets
 - ★ they have an IP address for each subnet
- ❑ Addressing for portable computers
 - ★ NAT: network address translation
 - ★ unregistered addresses (not unique throughout the network)
 - ★ special router does mapping

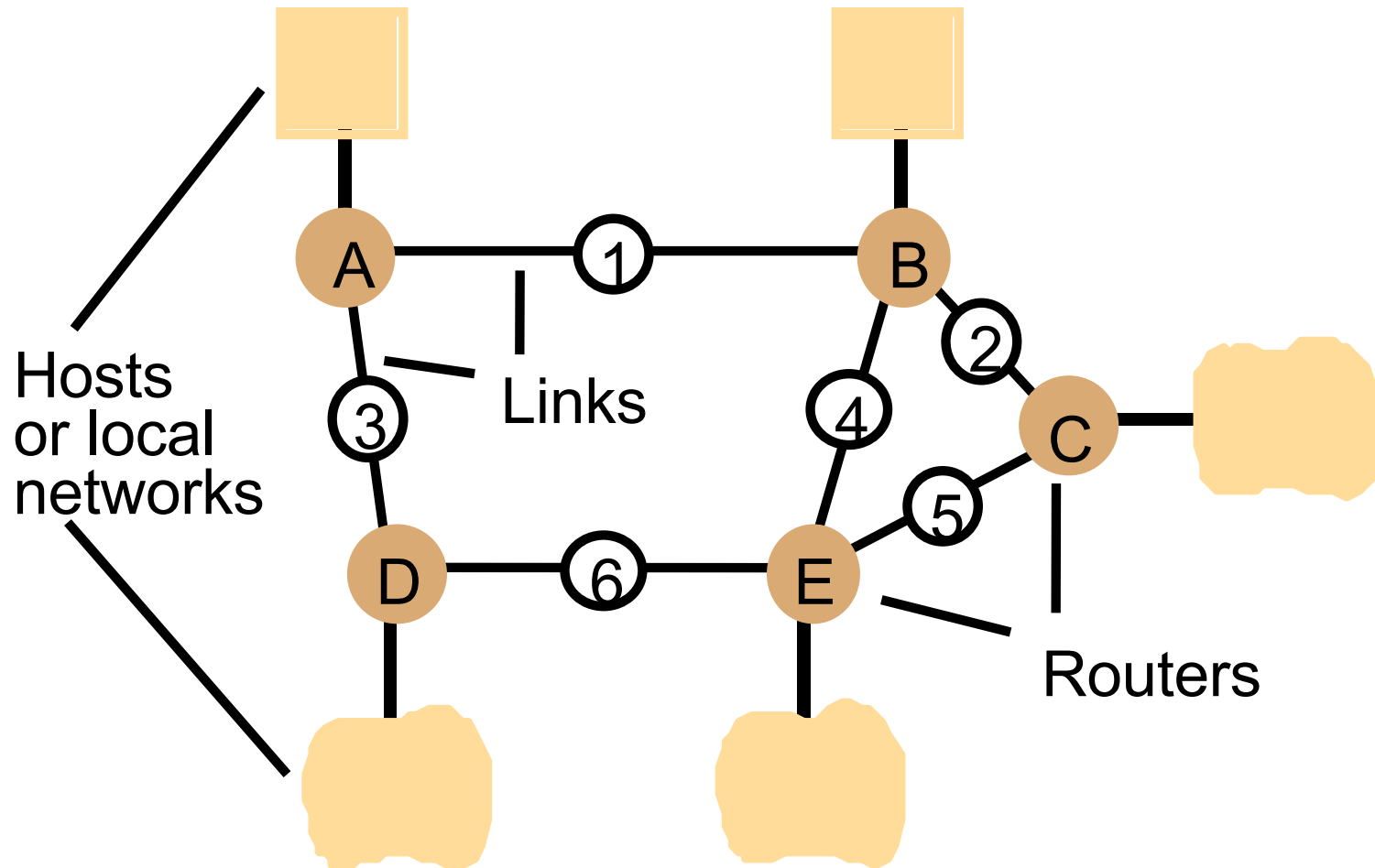
IP protocol

□ <https://en.wikipedia.org/wiki/IPv4#Header>

IPv4 header format

Offsets	Octet	0								1								2								3							
Octet	Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	Version				IHL				DSCP						ECN		Total Length															
4	32	Identification															Flags			Fragment Offset													
8	64	Time To Live							Protocol							Header Checksum																	
12	96	Source IP Address																															
16	128	Destination IP Address																															
20	160	Options (if IHL > 5)																															
24	192																																
28	224																																
32	256																																

Routing in a wide area network



Routing Table

<i>Routings from A</i>		
<i>To</i>	<i>Link</i>	<i>Cost</i>
A	local	0
B	1	1
C	1	2
D	3	1
E	1	2

<i>Routings from B</i>		
<i>To</i>	<i>Link</i>	<i>Cost</i>
A	1	1
B	local	0
C	2	1
D	1	2
E	4	1

<i>Routings from C</i>		
<i>To</i>	<i>Link</i>	<i>Cost</i>
A	2	2
B	2	1
C	local	0
D	5	2
E	5	1

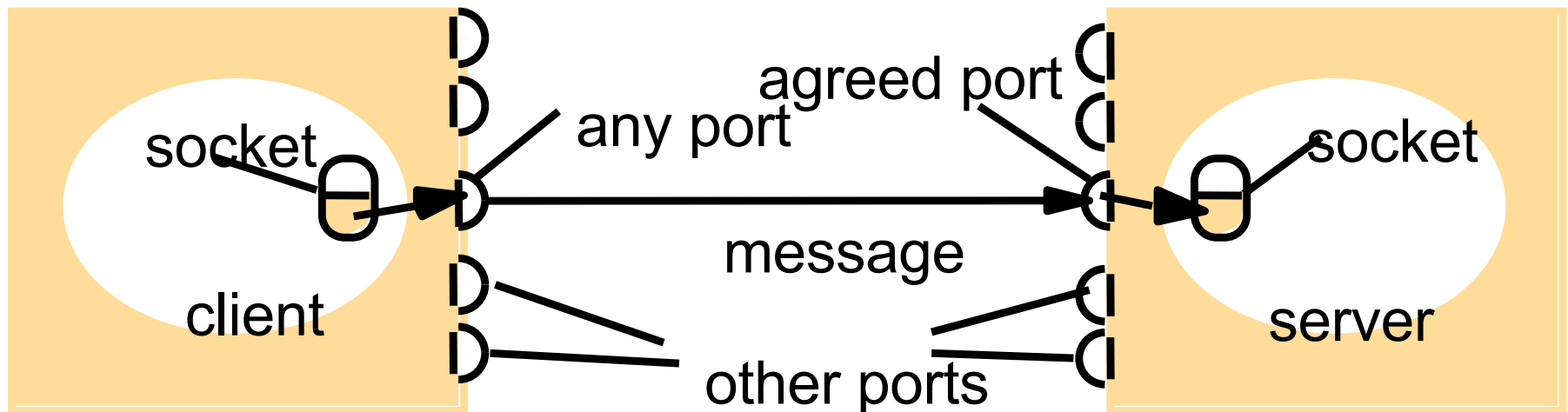
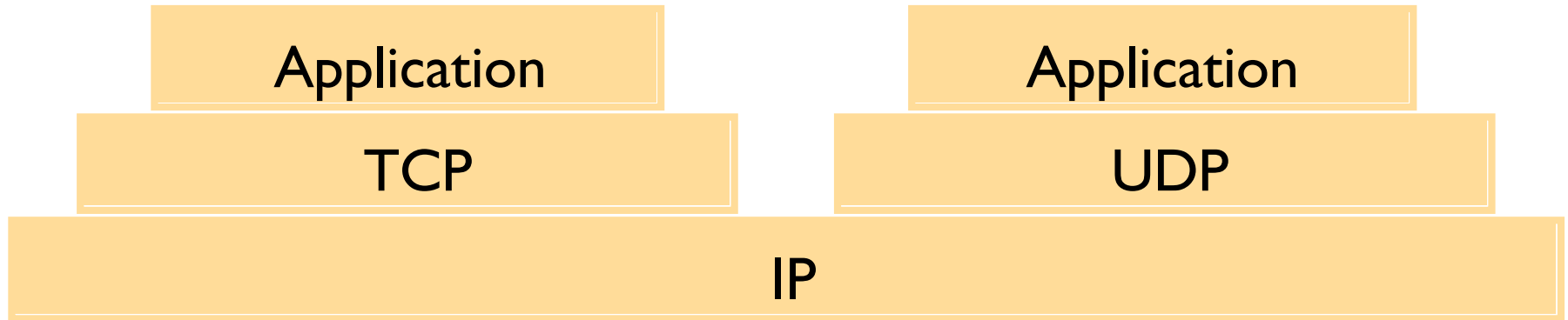
<i>Routings from D</i>		
<i>To</i>	<i>Link</i>	<i>Cost</i>
A	3	1
B	3	2
C	6	2
D	local	0
E	6	1

<i>Routings from E</i>		
<i>To</i>	<i>Link</i>	<i>Cost</i>
A	4	2
B	4	1
C	5	1
D	6	1
E	local	0

Issues

- ❑ Fast lookup of next hub
- ❑ maintenance
 - ★ if link broken, set cost to infinity
 - ★ send periodically routing table to others
 - others update their tables
- ❑ reasonable size
 - ★ geographically oriented IP addresses
 - ★ defaults

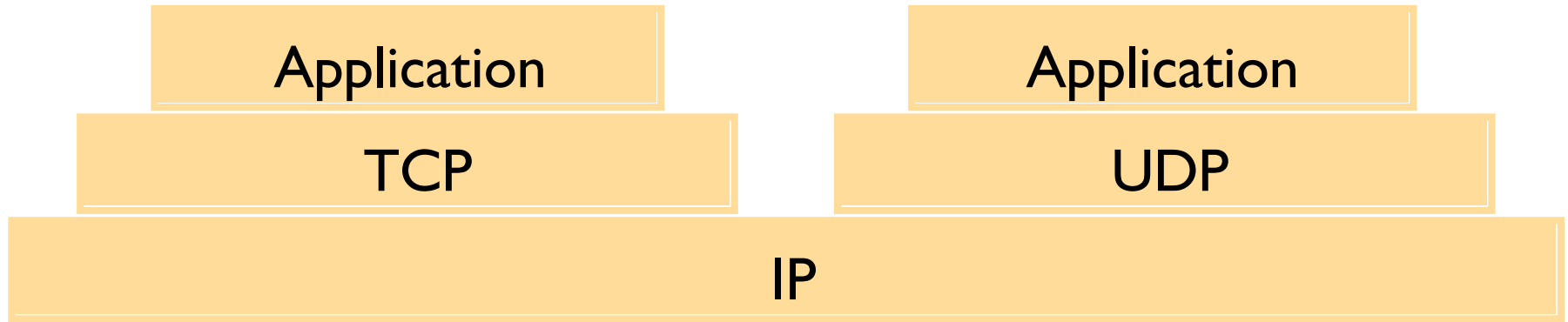
The programmers view



Internet address = 138.37.94.248

Internet address = 138.37.88.249

The programmers view



□ UDP

- ☆ thin software layer on top of IP which builds interface to application
- ☆ same functionality as IP: unreliable, best-effort, message-oriented
- ☆ checksums against message corruption: integrity
- ☆ domain name resolution: application can use logical name (mimi.cs.mcgill.ca) instead of IP address
- ☆ connection-less

UDP

- ❑ send(message byte string, message-length, IP receiver, port receiver)
 - ☆ UDP then adds IP sender, port sender
- ❑ receive(byte string into buffer, length, IP sender, IP port)
- ❑ Java
 - ☆ DatagramPacket: message, message length, IP address, port
 - IP address and port have different content for sending / receiving
 - ☆ DatagramSocket:
 - constructor with port as input (for server)
 - constructor without port as input (for client)
 - send and receive methods with DatagramPacket as argument

UDP

❏ https://en.wikipedia.org/wiki/User_Datagram_Protocol#UDP_datagram_structure

UDP datagram header

Offsets	Octet	0								1								2								3							
Octet	Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	Source port																Destination port															
4	32	Length																Checksum															

TCP

- ☆ a software layer on top of IP which builds interface to application
- ☆ checksums against message corruption: integrity (just like UDP)
- ☆ domain name resolution: application can use logical name (mimi.cs.mcgill.ca) instead of IP address (just like UDP)
- ☆ Provides **reliable message delivery** despite the possibility of message loss by the IP layer
- ☆ Provides **FIFO delivery** of messages (receive in the order they were sent)
- ☆ Stream-oriented: application does not need to think in “message”; can use different abstraction
- ☆ connection-oriented

TCP

❑ reliable:

- ☆ stream split into sequence of data segments (TCP messages) with sequence numbers
- ☆ acknowledgement scheme:
 - receiver sends to sender periodically
 - ▲ highest sequence number s so that all messages with sequence number $s' \leq s$ have been received + window size
 - Flow Control to avoid buffer overflow at receiver
 - ▲ sender can only send window size of data before next acknowledgement
 - retransmission of lost messages
 - ▲ sender keeps messages in output buffer until acknowledgment receives from receiver
 - ▲ if message not acknowledged within specified timeout, then it retransmits it.
 - ▲ duplicate detection using sequence numbers

TCP

❑ FIFO delivery

- ☆ sequence numbers allow detection of out-of-order messages and reorder them before delivering them to the application

TCP

❑ connection-oriented:

- ☆ Both processes first run a connection protocol
 - Handshake that both agree on building a communication channel
- ☆ Only after connection is established, application level messages can be sent in both directions

TCP

□ Overview

★ Client creates socket

★ Server has special socket object with associated port

- Server listens on socket for connection request message from client
- Clients makes connection request to this socket
- upon receiving request from Client C
 - ▲ create new socket with input and output stream only for communication with client

TCP

❑ stream-oriented

- ☆ both parties have input and output buffer
 - Associated with sockets
- ☆ Depending on programming language and API, an “application message” can be an object, a byte stream, a string, etc.
- ☆ The send appends this application message to the stream

TCP

□ Java

★ ServerSocket:

- constructor has port as input
- accept method returns new instance of Socket class which is already connected to client socket

★ Socket:

- one constructor (used by client) has as input name and port of server
 - ▲ creates socket with local port and connects to ServerSocket
- has associated input and output streams of certain types
 - ▲ (e.g., DataInputStream, ObjectInputStream, ...)
- input/output streams have methods: write/read

Java usage

- ❑ Assume an object variable myPerson of class Person with some attributes
- ❑ To exchange object of class Person via TCP
 - ☆ At sender
 - ☆ `ObjectOutputStream oos = new ObjectOutputStream(socket);`
 - ☆ `oos.writeObject(myPerson);`
 - ☆ At receiver
 - ☆ `ObjectInputStream ois = new ObjectInputStream(socket);`
 - ☆ `readPerson = (Person) ois.readObject();`

TCP

□ https://en.wikipedia.org/wiki/Transmission_Control_Protocol#TCP_segment_structure

TCP segment header

Offsets	Octet	0								1								2								3							
Octet	Bit	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
0	0	Source port																Destination port															
4	32	Sequence number																															
8	64	Acknowledgment number (if ACK set)																															
12	96	Data offset				Reserved 0 0 0			N S	C W R	E C E	U R G	A C K	P S H	R S T	S Y N	F I N	Window Size															
16	128	Checksum																Urgent pointer (if URG set)															
20	160	Options (if <i>data offset</i> > 5. Padded at the end with "0" bytes if necessary.)																															
...	...	...																															

Middleware layers

Applications

Request / Reply approaches: HTTP, RMI, RPC, WebService

External data representation

Middleware

Operating System/Basic Communication (TCP/UDP)

Data Representation in Messages

- ❑ Messages are a sequence of bytes -> data must be flattened
- ❑ Marshalling/serialization: transformation of a collection of data items into a form suitable for transmission
- ❑ Unmarshalling: transforming a message back into a collection of data items
- ❑ Common Mechanisms:
 - ❑ Java Serialization
 - ❑ XML / JSON
 - ❑ Google Protocol Buffer
 - ❑ CDR (Common Data Representation) / CORBA (Common Object Request Broker Architecture)

Java Declaration

```
Class Person implements Serializable {
```

```
    int id;
```

```
    string name;
```

```
    string place;
```

```
    int year;
```

```
    Public Person (int pid, string pname, string pplace, int pyear) {
```

```
        id = pid;
```

```
        name = pname
```

```
        place = pplace;
```

```
        year = pyear;
```

```
    }
```

```
    Person myPerson = new Person(12345, "Smith", "London", 1934);
```

Java Internal Representation

- ☆ Serialized binary format
- ☆ Contains information about data types

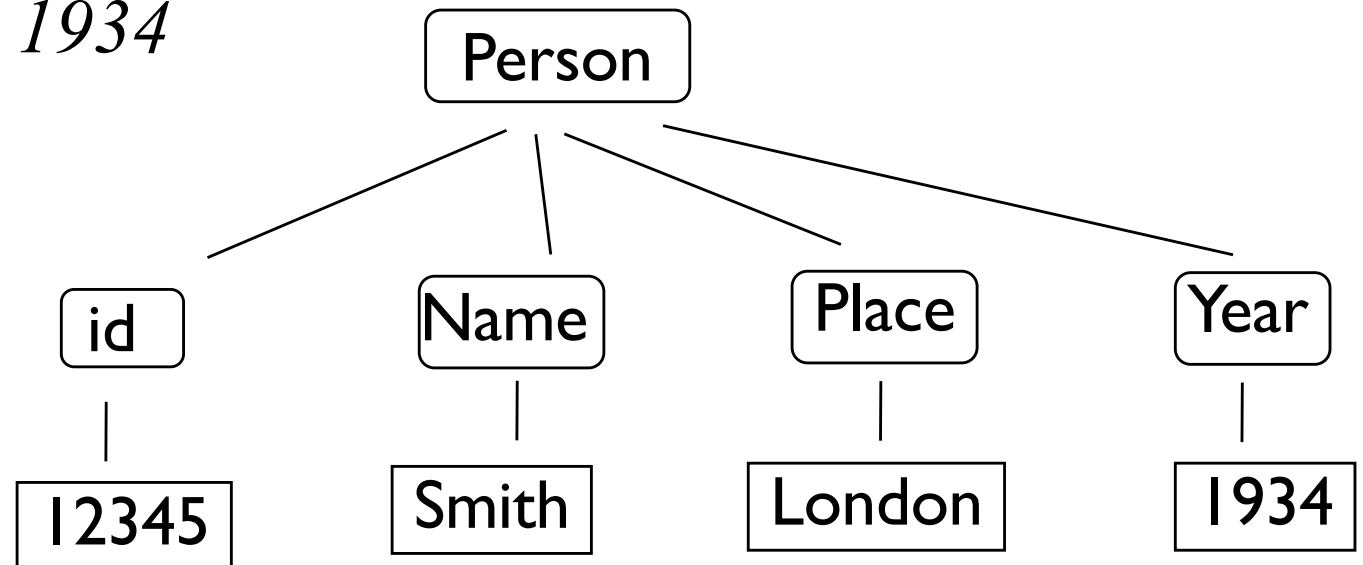
<i>Serialized values</i>			<i>Explanation</i>
Person	8-byte version number		h0 <i>class name, version number</i>
3	int year	java.lang.String name:	java.lang.String place: <i>number, type and name of instance variables</i>
1934	5 Smith	6 London	h1 <i>values of instance variables</i>

The true serialized form contains additional type markers; h0 and h1 are handles

JSON

- textual format to present structured data

```
{  
  "person": {  
    "id": 12345  
    "name": "Smith"  
    "place": "London"  
    "year": 1934  
  }  
}
```



XML

- textual format to present structured data

<person id="12345">

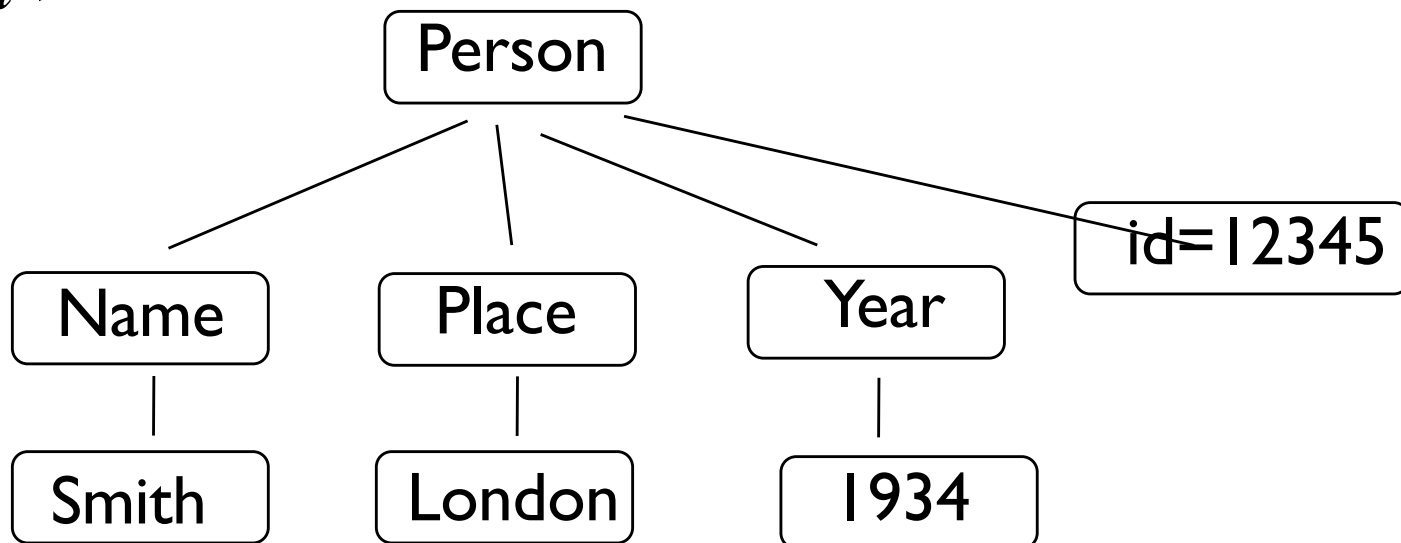
<name>Smith</name>

<place>London</place>

<year>1934</year>

<!-- a comment -->

</person >



XML

❑ elements

- ★ basic building blocks
- ★ have start and end tag which indicate the name of element
- ★ elements can be nested

❑ attributes

- ★ element can have attributes
- ★ attribute is name/value pair
- ★ embedded in start tag of element

❑ text

- ★ part of an element
- ★ represents the content

Json/XML

- ❑ Internal representation

 - ☆ Text / ascii

 - ☆ Human readable

 - ☆ Contains meta data and data

- ❑ There exist many parsers for JSON/XML

 - ❑ e.g., Java software for conversion between Java objects and XML/JSON .

Google Protocol Buffer

```
message Person {  
    required int32 id = 1;  
    required string name = 2;  
    optional string place = 3;  
    optional int32 year = 4;  
}  
  
Person person;  
person.set_name("John Doe");  
person.set_id(1234);  
person.set_year(1934);
```

Internal Representation

❑ Google Protocol Buffer

```
message Person {  
    required int32 id = 1;  
    required string name = 2;  
    optional string place = 3;  
    optional int32 year = 4;  
}  
  
Person person;  
  
person.set_id(1234);  
person.set_name("Smith");  
person.set_year(1934);
```

❑ Internal binary presentation (Simplified)

☆ Contains coded info about (1) data type (2) attribute (3) real data

0 1 1234

2 2 5 "Smith"

0 4 1934

Google Protocol Buffer

- ❑ Programming with Protocol Buffers in Java
 - ☆ Similar to TCP with input and output streams

Apache Arrow

CORBA Marshalling

- ❑ message contains only the data but not the information about data types
- ❑ programmer can specify the structs that are the content of messages in an IDL (interface definition language) file
 - ★ CORBA generates automatically marshalling and unmarshalling operations for these structs

CORBA's Data Representation

<i>index in sequence of bytes</i>	<i>4 bytes</i>	<i>notes on representation</i>
0–3	5	<i>length of string</i>
4–7	"Smit"	<i>'Smith'</i>
8–11	"h____"	
12–15	6	<i>length of string</i>
16–19	"Lond"	<i>'London'</i>
20–23	"on____"	
24–27	1934	<i>unsigned long</i>

The flattened form represents a *Person* struct with value:
 {'Smith', 'London', 1934}

Internal Representation

❑ XML/JSON

- ☆ Text / ascii
- ☆ Human readable
- ☆ Contains meta data and data

❑ Java

- ☆ Serialized binary format
- ☆ Contains information about data types

Serialized values

Person	8-byte version number		h0
3	int year	java.lang.String name:	java.lang.String place:
1934	5 Smith	6 London	h1

Explanation

class name, version number

*number, type and name of
instance variables*

values of instance variables

The true serialized form contains additional type markers; h0 and h1 are handles